

Python for Machine Learning

A Complete Learning Journey

Mayur Patil

August 22, 2026

Contents

Part I

Python Fundamentals

Chapter 1

Python Data Types

1.1 Why This Matters

In Machine Learning, your data will take many forms: numerical arrays, text labels, boolean masks, and configurations. Understanding the core Python data types is essential because applying the wrong operation to the wrong type (e.g., trying to add a string to an integer) will crash your pipeline. Knowing how to check types and understand "truthiness" will help you debug faster.

1.2 Explanation

1.2.1 1. Numeric Types

- **int**: Whole numbers. Python handles arbitrarily large integers automatically.
- **float**: Decimal numbers. Standard for ML model weights and probabilities.
- **complex**: Complex numbers (e.g., $3 + 4j$). Rarely used in standard ML, but important for signal processing.

1.2.2 2. Text and Binary Types

- **str**: Text data, enclosed in quotes. Immutable.
- **bytes** / **bytearray**: Raw byte data. Useful for reading images, audio, or serialized ML models.

1.2.3 3. Boolean and None

- **bool**: **True** or **False**. Used for logic and masking.
- **None**: A special constant representing the absence of a value (similar to **null** in other languages).

1.2.4 4. Collection Types (Brief intro)

- **list**: Ordered, mutable sequence `[1, 2, 3]`.
- **tuple**: Ordered, immutable sequence `(1, 2, 3)`.
- **set**: Unordered collection of unique items `{1, 2, 3}`.
- **dict**: Key-value pairs `{"a": 1, "b": 2}`.

1.2.5 5. `type()` and `isinstance()`

- `type(x)` returns the exact type of `x`.
- `isinstance(x, type)` checks if `x` is an instance of a specific type (or a tuple of types). **This is the preferred, professional way to check types.**

1.2.6 6. Truthiness

In Python, every object can be evaluated as a boolean.

- **Falsy:** `0`, `0.0`, `""` (empty string), `None`, `[]`, `{}`, `()`, `set()`.
- **Truthy:** Almost everything else!

1.2.7 7. Equality (`==`) vs Identity (`is`)

- `==` checks if the **values** are equal.
- `is` checks if they are the **exact same object in memory**.

1.3 Examples

1.3.1 Level 1 — Basic (Type Checking)

```

1 x = 3.14
2 print(type(x))                # Output: <class 'float'>
3 print(isinstance(x, float))    # Output: True
4 print(isinstance(x, (int, float))) # Output: True (checks if int OR float)

```

1.3.2 Level 2 — Intermediate (Truthiness)

```

1 empty_list = []
2 if empty_list:
3     print("This won't print because empty lists are falsy.")
4
5 if not empty_list:
6     print("This WILL print!") # Preferred way to check if a list is empty

```

1.3.3 Level 3 — Hard (Equality vs Identity with `None`)

```

1 a = None
2 b = None
3
4 # Because None is a singleton (only one exists in memory), BOTH are True.
5 print(a == b) # True
6 print(a is b) # True
7
8 list_1 = [1, 2]
9 list_2 = [1, 2]
10 print(list_1 == list_2) # True, they have the same values
11 print(list_1 is list_2) # False, they are different objects in memory!

```


1.4 Common Mistakes

WARNING:

1. Using `type(x) == list` instead of `isinstance(x, list)`. `isinstance` gracefully handles inheritance (subclasses), which is crucial when working with ML libraries that create custom object types.
2. Checking if a list is empty using `if len(my_list) == 0:`. The Pythonic way is simply `if not my_list:`.

ML CONNECTION:

When reading a CSV dataset using Pandas, missing values often start as `None` or `NaN` (Not a Number, which is a float). Knowing how to detect and handle these null types, and understanding how booleans act as masks (e.g., selecting all rows where `age < 30`), is the absolute foundation of data preprocessing.

1.5 Exercises

Exercise 1: Truthiness and Types Predict the output of the following code snippet:

```
1 data = ""
2 result = 100
3
4 if data:
5     result = result + 50
6 else:
7     result = result - 20
8
9 print(result)
```

Solution: The output is 80. An empty string evaluates to False in Python, triggering the else block.

Exercise 2: Debugging and Best Practices The following function tries to check if the input is a string and if it is not empty. However, it uses poor Python practices. Rewrite it using `isinstance()` and Pythonic truthiness.

```
1 def process_text(text):
2     if type(text) == str:
3         if len(text) > 0:
4             print("Processing:", text)
5         else:
6             print("Empty string provided.")
7     else:
8         print("Error: Input is not a string.")
```

Refactored Solution:

```
1 def process_text(text):
2     if isinstance(text, str):
3         if text:
4             print("Processing:", text)
5         else:
6             print("Empty string provided.")
7     else:
8         print("Error: Input is not a string.")
```